Universidad Tecnológica Nacional · FRC

Desarrollo Dirigido por Pruebas (TDD)

Cátedra de Ingeniería y Calidad de Software · Docentes: Judith Meles, Laura Covaro, Cecilia Massano · *"No pruebes el código. Diseñalo con pruebas."*

Definición

Test Driven Development (TDD) o Desarrollo Dirigido por Pruebas es una técnica de diseño y desarrollo en la que **no se escribe código de producción sin que antes exista una prueba automatizada que falle**. Involucra otras dos prácticas: *escribir las pruebas primero* (Test First Development) y *refactorización* (Refactoring); para escribir las pruebas se utilizan generalmente pruebas unitarias. Su punto clave es que **no es una técnica de testing sino una actividad de especificación y diseño**: el test se escribe antes que el código porque sirve para definir qué debe hacer el software.

1. Por qué TDD: el problema de probar al final

En el **desarrollo tradicional** el orden es Diseñar → Desarrollar → Probar: las pruebas unitarias se diseñan **después** de escribir el código y el aseguramiento de la calidad es **reactivo**. En **TDD** el orden pasa a ser Diseñar → Probar → Desarrollar: las pruebas se diseñan **antes** y el aseguramiento de la calidad se vuelve **proactivo**.

Dejar la prueba para el final no es solo una cuestión de orden; trae tres problemas concretos:

Problema	En qué consiste
Acoplamiento incidental	El código no se pensó para ser probado, por lo que sus piezas quedan entrelazadas y resulta muy difícil aislar una unidad para verificarla.
Deuda técnica y miedo al cambio	Sin una red de pruebas que avise si algo se rompe, modificar el código es riesgoso. El equipo deja de mejorarlo y la deuda se acumula.
Costo asintótico del error	Cuanto más tarde se detecta un defecto, más caro resulta corregirlo. El costo crece de forma acelerada a medida que avanza el ciclo de vida.

El cambio de mentalidad

TDD invierte el razonamiento: primero se define **el Qué** (qué debe hacer el software, expresado como un ejemplo concreto) y recién después se decide **el Cómo** (la implementación). Se pasa de *escribir código y cruzar los dedos* a *definir el éxito y construir hacia él*.

La metáfora del trinquete (Kent Beck)

Beck plantea que TDD es, ante todo, **una forma de gestionar el miedo durante la programación**. El miedo nos vuelve dubitativos, nos hace comunicarnos menos y evitar la retroalimentación. Programar es como subir un balde desde un pozo con una manivela: si el balde pesa, hace falta un *trinquete* que permita descansar sin que todo se venga abajo. **Los tests son los dientes de ese trinquete:** una vez que un test pasa, ese avance queda asegurado. Cuanto más difícil es el problema, más juntos deben estar los dientes, es decir, menos terreno debe cubrir cada test.

2. Las reglas de TDD

Las 2 reglas de Kent Beck

1. Escribir código nuevo **únicamente** si antes existe una prueba automatizada que falla.
2. **Eliminar la duplicación.**

De estas dos reglas simples Beck deriva el orden de las tareas de programación: **Rojo** → **Verde** → **Refactorizar**, el mantra de TDD.

Las 3 leyes de Robert C. Martin

1. No escribirás una línea de código de producción sin antes haber escrito un test unitario que falle.
2. No escribirás más código de prueba del necesario para provocar un fallo. Si el código ni siquiera compila, eso ya cuenta como fallo.
3. No escribirás más código de producción que el mínimo indispensable para que el test pase.

3. El corazón de TDD: el ciclo Red-Green-Refactor

Fase	Qué se hace	Enfoque
RED	Se declara la intención: “quiero que, al pasarle estos datos, el sistema me devuelva este resultado”. El test falla porque todavía no existe el código.	Declarativo (el *Qué*)
GREEN	Se escribe el **código mínimo** para que la prueba pase, sin importar que quede feo o chapucero.	Imperativo (el *Cómo*)
REFACTOR	Se modifica la estructura interna sin alterar el comportamiento externo: se elimina la duplicación y se mejora el diseño.	Mejora del diseño

Los pasos del ciclo

Escribir un test → hacer que el test compile → hacer que el test falle (Red) → hacer un cambio para que pase → correr el test y verlo pasar (Green) → quitar duplicaciones (Refactor) → y vuelta a empezar con la siguiente especificación.

Ejemplo: una calculadora

RED — se escribe primero el ejemplo; la clase todavía no hace nada y el test falla.

```
def test_sumar_dos_numeros_EXITO():
    calculadoraDePrueba = new Calculadora()          # Precondiciones
    resultado = calculadoraDePrueba.sumar(a=1, b=2)  # Pasos
    assert(resultado == 3)                           # Resultado

class Calculadora:
    def sumar(a, b): return "No implementado"
```

GREEN — código mínimo para que el test pase (patrón *Fake It*: se devuelve un valor fijo).

```
class Calculadora:
    def sumar(a, b): return 3
```

REFACTOR — se mejora la estructura interna sin cambiar el comportamiento (se tipan los parámetros).

```
class Calculadora:
    def sumar(a: Number, b: Number) -> Number: return 3
```

Nueva iteración — un segundo test con datos distintos fuerza la implementación general (patrón *Triangulación*).

```
def test_sumar_dos_numeros_EXITO():
    resultado = calculadoraDePrueba.sumar(a=3, b=2)
    assert(resultado == 5)

class Calculadora:
    def sumar(a: Number, b: Number) -> Number: return a + b
```

> Advertencia
>
> Si de las tres etapas del algoritmo se deja una afuera, no se está practicando TDD sino otra cosa. La que más tiende a olvidarse es la refactorización: **hacer TDD sin refactorizar es simplemente escribir código malo con pruebas.**

4. Patrones de TDD

Los patrones son estrategias, decisiones de diseño o atajos mentales que usa quien programa para **transicionar de una fase a otra dentro del ciclo**.

Patrones *red bar* — cómo escribir el test que falla	Patrones *green bar* — cómo hacerlo pasar
• **One Step Test:** escribir un solo test que verifique una pieza mínima de funcionalidad. • **Starter Test:** primer test que ayuda a explorar la API o el comportamiento deseado. • **Explanation Test:** escribir comentarios o preguntas para aclarar qué se espera que haga el código. • **Learning Test:** permite conocer APIs externas de las que no sabemos su comportamiento. • **Another Test:** añadir un test para un caso adicional o una variante del comportamiento. • **Regression Test:** test que reproduce un bug descubierto, para evitar regresiones futuras.	• **Fake It:** devolver un valor fijo o solución "falsa" solo para hacer pasar el test. • **Triangulación:** introducir un segundo test con datos diferentes para forzar una implementación más general. • **Obvious Implementation:** cuando la solución es trivial y obvia, implementarla directamente sin pasar por *Fake It*. • **One to Many:** patrón para resolver tests sobre colecciones de elementos.

Patrones de diseño de test	Otras familias
• **Test:** nombrar las pruebas con la palabra *test* y la clase a probar. • **Isolated Test:** mantener los tests totalmente independientes entre sí. • **Test List:** escribir primero la lista de tests que se van a realizar. • **Test First:** escribir primero la prueba. • **Assert Test:** escribir primero la confirmación y luego la prueba. • **Test Data:** usar datos de prueba fáciles de leer y seguir. • **Evident Data:** incluir los resultados esperados y reales dentro del test, y que la relación entre los datos sea evidente.	• Patrones de testing • Patrones de diseño • **xUnit Patterns:** *assertion*, *fixture*, *external fixture*, *test method*, *exception test*, *all tests*.

5. Refactoring

Es una **forma disciplinada de introducir cambios reduciendo la posibilidad de introducir defectos**. Según Martin Fowler, refactorizar es modificar el diseño sin alterar su comportamiento: es una transformación que **preserva la estructura externa y mejora la estructura interna**. Refactorizar no es lo mismo que reescribir; reescribir es más general.

¿Qué caracteriza a un refactoring?

- No cambia el comportamiento observable (todas las pruebas siguen pasando).
- Elimina la duplicación o la complejidad innecesaria.
- Mejora la calidad del software y lo hace más simple de entender.
- Flexibiliza el código y lo hace más fácil de cambiar.

¿Por qué es necesario?

- Evitar el "deterioro del diseño" y limpiar el desorden del código.
- Incrementar la legibilidad y la comprensibilidad.
- Encontrar errores y reducir el tiempo de depuración.
- Incorporar el aprendizaje que vamos haciendo sobre la aplicación.

¿Cómo se hace?

Asegurarse de que las pruebas pasen (TDD) → encontrar un código que "huela mal" → determinar cómo simplificarlo → hacer las simplificaciones → ejecutar las pruebas para asegurarse de que sigan pasando. La clave es avanzar **en pasos muy pequeños**: un cambio, se corren todos los tests y, si todo sigue funcionando, otro cambio. Se piensa en global pero se actúa en local.

6. Más allá del ciclo unitario

ATDD: el Qué y no el Cómo

Cuando la especificación se escribe a nivel de **historia de usuario o de aceptación** en lugar de a nivel unitario, hablamos de **ATDD** (Acceptance Test Driven Development). El principio es que el Dueño del Producto dice *qué* necesita, no *cómo* implementarlo. La técnica consiste en reemplazar descripciones ambiguas por **ejemplos precisos** que el cliente pueda validar como verdaderos o falsos, lo que obliga a explicitar decisiones de negocio que de otro modo terminaría tomando quien desarrolla.

Tipos de test

Tipo	Alcance
Unitario	Prueba un único comportamiento de un método de una clase, de forma aislada del resto del sistema.
De integración	Verifica que distintas partes del sistema se unan bien (por ejemplo, que la lógica de negocio entienda datos reales de la base). Granularidad más gruesa y más frágil, por eso suelen ser menos.
De aceptación	Expresa un requisito de negocio; es el que valida el cliente. Es el nivel propio de ATDD.

Un test unitario debe ser **atómico** (prueba la mínima funcionalidad posible), **independiente** (no depende de otros tests ni de un orden), **inocuo** (no altera el estado del sistema: ejecutarlo una vez o veinte da el mismo resultado) y **rápido**. Si no cumple estas premisas no es un test unitario, aunque se ejecute con una herramienta xUnit. Las mismas características se agrupan bajo las siglas **F.I.R.S.T.**: *Fast, Independent, Repeatable, Small, Transparent*.

> Las tres partes del test: AAA
>
> **Arrange** (preparar): se arma el escenario y se crean las variables necesarias. — **Act** (actuar): se hace la llamada al código que se quiere probar, el SUT (*Subject Under Test*). — **Assert** (afirmar): se afirma sobre el resultado que las expectativas se cumplen; si no se cumplen, el framework lo marca en rojo. La preparación común a todos los tests de una clase suele ir en el método *SetUp*, y la limpieza posterior en *TearDown*.

Dobles de prueba

Para aislar el SUT se usan **dobles**, en el mismo sentido que los dobles de riesgo en el cine: se hacen pasar por un colaborador sin serlo. Según la nomenclatura de Meszaros y Fowler:

Doble	Qué hace
Dummy	Se pasa como argumento pero nunca se usa realmente; sirve para rellenar listas de parámetros.
Fake	Tiene una implementación que funciona, pero toma un atajo que la hace inapropiada para producción (por ejemplo, una base de datos en memoria).
Stub	Devuelve respuestas predefinidas a las llamadas hechas durante el test. Simula respuestas a consultas.
Mock	Objeto preprogramado con expectativas sobre cómo deben recibirse las llamadas. **Valida el comportamiento de la colaboración**, no solo el estado.

El stub hace el test menos frágil pero aporta menos información sobre la colaboración entre objetos. Hay que usarlos con criterio: abusar de los mocks vuelve el test difícil de leer y **frágil**, porque cualquier cambio pequeño en el SUT lo rompe.

7. Beneficios y límites

Beneficios

- Código más **robusto, predecible y limpio**, que permite saber cuándo está terminado.
- Código más **tolerante al cambio** y más seguro.
- Reduce la duplicación: si el test pasa, el código ya existe.
- Más **barato de mantener** al ser más entendible.
- Con la práctica **acelera** la velocidad de desarrollo y habilita el despliegue continuo.
- Asegura **cobertura de prueba**, lo que conlleva código de calidad.
- Obliga a pensar cómo se usa el componente, mejorando el **diseño de las API**.
- Promueve componentes con **alta cohesión y bajo acoplamiento**.

Límites y advertencias

- Beck reconoce tareas que **no pueden dirigirse solo por tests**: la **seguridad**, que depende también del juicio humano sobre los métodos usados, y la **concurrencia**, cuyos problemas sutiles no se reproducen de forma confiable al correr el código.
- El abuso de **mocks** genera tests frágiles y difíciles de leer.
- Requiere entrenamiento: la esencia es sencilla, pero al principio siempre se escribe más código del que hace falta.
- El tamaño del proyecto no condiciona su aplicabilidad: un proyecto grande no es más que una agrupación de subproyectos chicos. La clave está en saber dividir y priorizar.

Bibliografía

Presentaciones de la cátedra

• Meles, J.; Covaro, L.; Massano, C. — *Test Driven Development (TDD)* — Cátedra de Ingeniería de Software, UTN-FRC.
• *Desarrollo Dirigido por Pruebas (TDD): No pruebes el código, diseñalo con pruebas* (v1.1) — Cátedra de Ingeniería de Software, UTN-FRC.

Material bibliográfico

• Beck, K. (2002) — *Test-Driven Development By Example* — Three Rivers Institute / Addison-Wesley.
• Blé Jurado, C. y colaboradores (2010) — *Diseño Ágil con TDD*, 1ª edición — www.iExpertos.com, Licencia Creative Commons.

Fuentes complementarias citadas por la cátedra

• Martin, R. C. — *Clean Code* (origen de las tres leyes de TDD).
• Huett, B. — artículo *Embracing Test Driven Development*.
• Fowler, M. — *Refactoring* y el artículo *Mocks Aren't Stubs*.
• Meszaros, G. — *xUnit Test Patterns* (nomenclatura de los dobles de prueba).